

EFFICIENT CP ROUNDING USING ALTERNATING LEAST SQUARES WITH QR DECOMPOSITION*

ALEX ZHANG[†], BHISHAM DEV VERMA[†], JAN VAN LENT[‡], AND GREY BALLARD[†]

Abstract. The CANDECOMP/PARAFAC (CP) decomposition is widely used for analyzing multidimensional data, and the alternating least squares (CP-ALS) algorithm is a common method for its computation. CP rounding is the problem of computing a lower-rank CP decomposition of an input already in a higher-rank CP format. While the normal equations (NE) approach in CP-ALS is efficient for the CP rounding problem and frequently used, it becomes unstable in the presence of ill-conditioned subproblems. This paper presents a new QR-based CP-ALS method for CP rounding that preserves both numerical stability and computational efficiency. Our experiments show that the proposed method offers significant speedup over a previous QR-based approach and the Tensor Toolbox’s NE-based implementation, particularly for higher-order tensors. Furthermore, our approach demonstrates a marked reduction in error for ill-conditioned problems, with error reductions several orders of magnitude smaller compared to the NE-based method, while achieving faster convergence and more accurate solutions. By using a more numerically stable approach, we can solve more problems in reduced working precision, which enables further reduction in time to solution.

1. Introduction. The CANDECOMP/PARAFAC or canonical polyadic (CP) decomposition for tensors is a popular tool for analyzing and interpreting latent patterns that may be present in multidimensional data. A rank- r CP decomposition of a d -way tensor is a sum of r rank-one components, each of which is a d -way vector outer product. One of the most popular methods used to compute a CP decomposition is the alternating least squares (CP-ALS) algorithm, which solves a series of linear least squares problems. The normal equations are typically used to solve these linear least squares problems within CP-ALS because of the structure of the coefficient matrix [5]. However, this approach is sensitive to roundoff error in the case of ill-conditioned coefficient matrices. Minster et al. [6] propose a more numerically stable approach using QR decomposition that also exploits the structure of the coefficient matrices within the linear least squares problems. One drawback of their more stable approach is that the computational cost is exponential in d , the number of modes of the tensor. This cost is often not the bottleneck when computing CP decompositions of input tensors in dense format. However, in the context of CP-Rounding, or computing a lower-rank CP decomposition of an input already in CP format, the exponential cost becomes the overwhelming bottleneck and renders the algorithm infeasible for large problems. In this paper, we propose a new method based on the QR decomposition that is just as numerically stable and reduces the costs from exponential to linear in d .

As we describe in section 2, in the context of CP-ALS, the coefficient matrix of each linear least squares problem is a Khatri-Rao product of matrices, which is a column-wise Kronecker product. ?? details the various methods for exploiting the structure of these coefficient matrices, including via the normal equations (see subsection 3.1 for more details). In order to efficiently compute a QR decomposition of a matrix with this structure, we perform a sequence of QR decompositions of smaller

*Submitted to the editors DATE.

Funding: This work is supported by the National Science Foundation under Grant Numbers CCF-1942892 and OAC-2106920.

[†]Wake Forest University, Winston-Salem, NC 27109 USA (zhanl21@wfu.edu, vermadb@wfu.edu, ballard@wfu.edu).

[‡]University of the West of England, Bristol, UK (jan.vanlent@uwe.ac.uk).

matrices. We illustrate the key idea with an example. Considering a Khatri-Rao product of three matrices each with r columns, we exploit the following identities, where each computed QR decomposition is denoted using underbraces:

$$\begin{aligned}
 (1.1) \quad \mathbf{A} &= \underbrace{\mathbf{A}_1}_{\mathbf{Q}_1 \mathbf{R}_1} \odot \underbrace{\mathbf{A}_2}_{\mathbf{Q}_2 \mathbf{R}_2} \odot \underbrace{\mathbf{A}_3}_{\mathbf{Q}_3 \mathbf{R}_3} \\
 (1.2) \quad &= (\mathbf{Q}_1 \otimes \mathbf{Q}_2 \otimes \mathbf{Q}_3) \underbrace{(\mathbf{R}_1 \odot \mathbf{R}_2 \odot \mathbf{R}_3)}_{\hat{\mathbf{Q}}_{12} \hat{\mathbf{R}}_{12}} \\
 (1.3) \quad &= (\mathbf{Q}_1 \otimes \mathbf{Q}_2 \otimes \mathbf{Q}_3) (\hat{\mathbf{Q}}_{12} \otimes \mathbf{I}_r) \underbrace{(\hat{\mathbf{R}}_{12} \odot \mathbf{R}_3)}_{\hat{\mathbf{Q}}_{123} \mathbf{R}}.
 \end{aligned}$$

Following [6], we start with a (compact) QR decomposition of each individual factor in (1.1), as the Kronecker product of the orthonormal factors can be factored out of the Khatri-Rao product as shown in (1.2). Note that the Khatri-Rao product of triangular matrices is not triangular, so we do not yet have a QR decomposition of $\mathbf{A}$. As we detail in subsection 3.2, the previous approach forms the explicit Khatri-Rao product of all triangular factors and performs a large QR decomposition with cost that is exponential in the number of factors. Instead, we propose using a pairwise approach. In (1.2) we form an explicit Khatri-Rao product of the first two triangular factors, compute a QR decomposition, and then factor out the orthonormal factor, leaving a Khatri-Rao product with one fewer factor matrix. In (1.3), $\hat{\mathbf{Q}}_{12}$ is $r^2 \times r$ and $\mathbf{I}_r$ is the $r \times r$ identity matrix. Finally, we form an explicit Khatri-Rao product of the remaining two triangular factors and compute the last QR decomposition. This achieves a QR decomposition of the matrix $\mathbf{A}$ where the orthonormal factor is a product of implicitly represented orthonormal matrices. We exploit the implicit structure when applying its transpose to the target matrix of the least squares problem, as the target matrix has similar structure in the case of CP-Rounding. The cost of the pairwise approach is linear in the number of factors; full details are given in subsection 3.3.

Section 4 presents experimental results to validate the complexity analysis and numerical stability properties of our proposed algorithm. We compare our proposed algorithm with the normal equations method and with the previously proposed QR-based algorithm to show that it is as computationally efficient as the normal equations and as numerically stable as the QR-based algorithm. We use a sine-of-sums tensor [2] to illustrate the differences among the algorithms, as the example yields ill-conditioned least squares problems that expose the numerical instability of the normal equations and can be scaled to large numbers of modes. Our results show a $14\times$ speedup over the previous QR-based algorithm on a single linear least squares solve for a problem with 7 Khatri-Rao factors. We also show that our implementation of CP-ALS using QR is faster than the implementation of CP-ALS in the MATLAB Tensor Toolbox [1], which is based on the normal equations, because our implementation uses memoization to avoid recomputation of temporary quantities. We observe up to a $4\times$ speedup over the normal equations based CP-ALS on a 9-way tensor. For the sine-of-sums tensors, we show that the normal equations approach can break down completely in the presence of ill-conditioning, while our method maintains small relative errors. We also illustrate for synthetic data the benefits of trading off accuracy and running time using reduced working precision: our proposed approach is able to run even faster using single precision while maintaining the same accuracy as the normal equations approach using double precision. We summarize our conclusions in section 5 and

87 discuss future directions based on the proposed method.

88 2. Background.

89 **2.1. CP Decomposition.** Given a d -way tensor $\mathcal{X} \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$, its CP
90 decomposition $\mathcal{K}$ of rank $r \in \mathbb{N}$ can be represented as

$$91 \quad (2.1) \quad \mathcal{X}(i_1, i_2, \dots, i_d) \approx \mathcal{K}(i_1, i_2, \dots, i_d) = \sum_{j=1}^r \lambda(j) \mathbf{A}_1(i_1, j) \mathbf{A}_2(i_2, j) \dots \mathbf{A}_d(i_d, j)$$

92 for all $(i_1, i_2, \dots, i_d) \in [n_1] \otimes [n_2] \otimes [n_3] \otimes \dots \otimes [n_d]$ where $\mathbf{A}_k \in \mathbb{R}^{n_k \times r}$ is a factor
93 matrix for each $k \in [d]$ and $\boldsymbol{\lambda} \in \mathbb{R}^r$ is a vector of weights.

We can represent $\mathcal{K}$ in shorthand CP notation as $\mathcal{K} = \llbracket \boldsymbol{\lambda}; \mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_d \rrbracket$. There exists ambiguity in the weighting of the columns of the factor matrices and $\boldsymbol{\lambda}$ vector. For simplicity of presentation, we drop the weight vector $\boldsymbol{\lambda}$ throughout and use the representation

$$\mathcal{K} = \llbracket \mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_d \rrbracket.$$

A tensor unfolding is a reshaping of the tensor elements into a matrix; mode-wise unfoldings map tensor fibers of a particular mode to columns of a matrix. In the case of CP-format tensors like $\mathcal{K}$, the k th mode-wise unfolding can be expressed in terms of the factor matrices:

$$\mathbf{K}_{(k)} = \mathbf{A}_k (\mathbf{A}_N \odot \dots \odot \mathbf{A}_{k+1} \odot \mathbf{A}_{k-1} \odot \dots \odot \mathbf{A}_1)^\top,$$

94 where $\odot$ denotes the Khatri-Rao product.

95 We use the following useful property of Khatri-Rao products:

PROPERTY 1. *Given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}$ with compatible dimensions,*

$$\mathbf{AB} \odot \mathbf{CD} = (\mathbf{A} \otimes \mathbf{C})(\mathbf{B} \odot \mathbf{D}).$$

96 **2.2. Linear Least Squares Problem.** In mathematical form, a generic linear
97 least squares problem is given by

$$98 \quad (2.2) \quad \min_{\mathbf{X}} \|\mathbf{B} - \mathbf{XA}^\top\|_F.$$

99 Note that the coefficient matrix $\mathbf{A}$ appears on the right of the variable matrix, which
100 is the opposite of the standard form. We use this form to relate to the corresponding
101 least squares problem that arises in the context of computing a CP decomposition.

102 Several different techniques can be used to solve this problem. The two we con-
103 sider here are via the Normal Equations (NE) and by QR decomposition. The NE
104 specify a (square) linear system of equations and are derived from setting the gradient
105 of the objective function from (2.2) to zero, and they involve the Gram of $\mathbf{A}$ as well
106 as the product $\mathbf{BA}$:

$$107 \quad (2.3) \quad \mathbf{XA}^\top \mathbf{A} = \mathbf{BA}.$$

108 The NE can be solved via Cholesky factorization of $\mathbf{A}^\top \mathbf{A}$ and triangular solves. The
109 cost of NE depends on the size of $\mathbf{A}$ and $\mathbf{B}$.

110 In the QR decomposition approach, we first compute the compact QR decompo-
111 sition $\mathbf{A} = \mathbf{QR}$ (so that $\mathbf{Q} \in \mathbb{R}^{m \times n}$) and then solve the triangular system

$$112 \quad (2.4) \quad \mathbf{XR}^\top = \mathbf{BQ}.$$

For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{p \times m}$, the time complexity of each step of Normal Equations and QR decomposition is given in Table 1. Note that the cost of the QR decomposition assumes that Householder QR is used in the factorization and that the $\mathbf{Q}$ matrix is formed explicitly. If p is smaller than n , then it is more efficient to leave $\mathbf{Q}$ in implicit Householder form, reducing the cost of QR to $2mn^2 - 2/3 n^3$. In this case, we apply $\mathbf{Q}$ to $\mathbf{B}$ using Householder transformations, which increases the cost of computing $\mathbf{BQ}$ to $4mnp$.

Normal Equations		QR Decomposition	
Component	Number of flops	Component	Number of flops
Compute Gram	mn^2	Compute QR	$4mn^2 - 4/3 n^3$
Compute $\mathbf{BA}$	$2mnp$	Compute $\mathbf{BQ}$	$2mnp$
Cholesky factorization	$1/3 n^3$	N/A	
Back solves	$2pn^2$	Back solves	pn^2

TABLE 1

Breakdown of time for using NE and QR to solve (2.2) for $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{p \times m}$

If $\mathbf{B}$ has few columns ($p \ll n$), NE is cheaper because of the smaller coefficient in the leading order cost (mn^2 vs $2mn^2$). However, when $\mathbf{B}$ has many columns ($p \gg n$), the largest cost is computing $\mathbf{BA}$ for NE and $\mathbf{BQ}$ for QR factorization, which have leading order cost with the same constant ($2mnp$).

Using QR is more numerically stable than NE. Let $\kappa = \sigma_1(\mathbf{A})/\sigma_n(\mathbf{A})$ be the standard condition number of $\mathbf{A}$. If we use QR to solve a least squares problem with a single right hand side; i.e. $\min \|\mathbf{Ax} - \mathbf{b}\|$, then from [8, Eq. (19.2)] the relative forward error satisfies

$$(2.5) \quad \frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} = O\left(\left(\kappa + \frac{\kappa^2 \tan \theta}{\eta}\right) \epsilon_{mach}\right),$$

where $\tilde{\mathbf{x}}$ is the computed solution, $\mathbf{x}$ is the exact solution, θ is the angle between $\mathbf{Ax}$ and $\mathbf{b}$, and $\eta = \|\mathbf{A}\| \|\mathbf{x}\| / \|\mathbf{Ax}\|$. However, if we use the NE approach, then by [8, Eq. (19.3)] the forward error guarantee can be much larger:

$$(2.6) \quad \frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} = O\left(\kappa^2 \epsilon_{mach}\right).$$

Because of the numerical sensitivity introduced by using the NE approach, the QR approach is much more accurate when κ is large (e.g., on the order of $\epsilon_{mach}^{-1/2}$ or larger) and θ is small.

2.3. CP-ALS.

2.3.1. CP Optimization Problem. When computing a rank- r CP approximation $\mathcal{K}$ for a d -way tensor $\mathcal{X}$, we seek to minimize $\|\mathcal{X} - \mathcal{K}\|^2$, where the tensor norm generalizes the vector 2-norm or the matrix Frobenius norm. Expressed using (2.1), we have

$$\min_{\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_d} \sum_{i_1=1}^{n_1} \cdots \sum_{i_d=1}^{n_d} \left(\mathcal{X}(i_1, i_2, \dots, i_d) - \sum_{j=1}^r \mathbf{A}_1(i_1, j) \mathbf{A}_2(i_2, j) \cdots \mathbf{A}_d(i_d, j) \right)^2.$$

However, this is a nonlinear, nonconvex optimization problem and no closed form solution exists. We must resort to using iterative methods to approximate a solution [5].

2.3.2. CP-ALS. One of the most effective algorithms for computing a CP decomposition of a tensor is called Alternating Least Squares (CP-ALS). The algorithm works as a block coordinate descent method, where each block of variables is a factor matrix. That is, it alternates over the factor matrices, holding all but one fixed and computing the optimal solution for the variable matrix.

We can solve each subproblem directly because it is a linear least squares problem. In updating the k th factor matrix, we expose the linear least squares problem by unfolding $\mathcal{X}$ and $\mathcal{K}$ along the k th mode:

$$\begin{aligned} \min_{\mathbf{A}_k} \|\mathcal{X} - \mathcal{K}\|^2 &= \min_{\mathbf{A}_k} \|\mathbf{X}_{(k)} - \mathbf{K}_{(k)}\|_F^2 \\ &= \min_{\mathbf{A}_k} \|\mathbf{X}_{(k)} - \mathbf{A}_k(\mathbf{A}_d \odot \cdots \odot \mathbf{A}_{k+1} \odot \mathbf{A}_{k-1} \odot \cdots \odot \mathbf{A}_1)^\top\|_F^2 \\ (2.7) \quad &= \min_{\mathbf{A}_k} \|\mathbf{X}_{(k)} - \mathbf{A}_k \mathbf{Z}_k^\top\|_F^2, \end{aligned}$$

where $\mathbf{Z}_k = \mathbf{A}_d \odot \cdots \odot \mathbf{A}_{k+1} \odot \mathbf{A}_{k-1} \odot \cdots \odot \mathbf{A}_1$. Note that the coefficient matrix $\mathbf{Z}_k^\top$ appears on the right of the variable matrix, matching the generic formulation in (2.2). All of the algorithms we consider in this report are based on CP-ALS, but they differ in how they solve the linear least squares subproblems specified in (2.7).

2.3.3. CP-Rounding. In many applications, we wish to compute a CP decomposition of a tensor that is already in CP format, sometimes referred to as a Kruskal tensor. That is, we wish to compute a lower-rank representation of the input tensor, sometimes referred to as a CP-Rounding. If $\mathcal{X} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ is a CP-format tensor with rank s , it can be written as

$$\mathcal{X}(i_1, i_2, \dots, i_d) = \sum_{j=1}^s \mathbf{B}_1(i_1, j) \mathbf{B}_2(i_2, j) \dots \mathbf{B}_d(i_d, j)$$

with shorthand notation $\mathcal{X} = [\mathbf{B}_1, \dots, \mathbf{B}_d]$.

In the case the input tensor is in CP format, we can then utilize its special mode- k unfolding representation

$$\mathbf{X}_{(k)} = \mathbf{B}_k(\mathbf{B}_d \odot \cdots \odot \mathbf{B}_{k+1} \odot \mathbf{B}_{k-1} \odot \cdots \odot \mathbf{B}_1)^\top$$

when solving the linear least squares problem given in (2.7). That is, the least squares subproblem is given by

$$(2.8) \quad \min_{\mathbf{A}_k} \|\mathbf{B}_k \mathbf{Y}_k^\top - \mathbf{A}_k \mathbf{Z}_k^\top\|_F^2,$$

where $\mathbf{Y}_k = \mathbf{B}_d \odot \cdots \odot \mathbf{B}_{k+1} \odot \mathbf{B}_{k-1} \odot \cdots \odot \mathbf{B}_1$. This structure can be exploited in different ways depending on how the least squares problem is solved.

3. Algorithms. **GB:** need to make sure we say that we assume no memoization in Algs 1,3,5 (with dense input) and we always form an explicit Q with QRs in Algs 3,4,5,6 because it's cheaper to do it that way

We now present three versions of the CP-ALS algorithm and compare their complexities. Subsection 3.1 presents the standard approach based on the normal equations, subsection 3.2 presents an existing approach that uses QR decomposition, and our proposed algorithm is detailed in subsection 3.3. We consider general d -way tensors with dimensions $n_1 \times \cdots \times n_d$, and we use r to denote the rank of the CP approximation. In the case of CP-format input, we use s to denote the input rank. For

complexity analysis, we use notation for arithmetic average ($\bar{n}$) as well as geometric mean ($\hat{n}$) of the tensor dimensions, which means the number of dense input tensor entries is $\hat{n}^d$ and the number of entries in the CP decomposition is $d\bar{n}r$. Summaries of the per-iteration complexity of each algorithm are given in Table 2 (for dense input) and Table 3 (for CP format input).

Normal Equations (Algorithm 3.1)		Explicit QR (Algorithm 3.3)		Pairwise QR (Algorithm 3.5)	
Component	Cost	Component	Cost	Component	Cost
MTTKRP [Line 9]	$2d\hat{n}^d r$	Multi-TTM [Line 8]	$2d\hat{n}^d r$	Multi-TTM [Line 8]	$2d\hat{n}^d r$
Compute Gram [Lines 8 and 11]	$d\bar{n}r^2 + d^2 r^2$	QR on factor matrices [Line 12]	$4d\bar{n}r^2$	QR on factor matrices [Line 17]	$4d\bar{n}r^2$
		Compute $\mathbf{Q}_0$ [Line 9]	$4dr^{d+1}$	Compute pairwise $\hat{\mathbf{Q}}$ [Line 12]	$4d^2 r^4$
		Apply $\mathbf{Q}_0$ [Line 10]	$2d\bar{n}r^d$	Apply pairwise $\hat{\mathbf{Q}}$ [Line 14]	$2d\bar{n}r^d$
NE Solve [Line 10]	$2d\bar{n}r^2 + \frac{1}{3}dr^3$	Back Solve [Line 11]	$d\bar{n}r^2$	Back Solve [Line 16]	$d\bar{n}r^2$
Leading term	$2d\hat{n}^d r$	Leading term	$2d\hat{n}^d r$	Leading term	$2d\hat{n}^d r$

TABLE 2

Per-iteration complexity of CP-ALS algorithms for dense input, reporting only the dominant terms for each component.

Normal Equations (Algorithm 3.2)		Explicit QR (Algorithm 3.4)		Pairwise QR (Algorithm 3.6)	
Component	Cost	Component	Cost	Component	Cost
MTTKRP [Lines 10 and 13]	$4d\bar{n}rs + d^2 rs$	Multi-TTM [Lines 11 and 14]	$4d\bar{n}rs$	Multi-TTM [Lines 17 and 20]	$4d\bar{n}rs$
Compute Gram [Lines 9 and 12]	$d\bar{n}r^2 + d^2 r^2$	QR on factor matrices [Line 13]	$4d\bar{n}r^2$	QR on factor matrices [Line 19]	$4d\bar{n}r^2$
		Compute $\mathbf{Q}_0$ [Line 9]	$4dr^{d+1}$	Compute pairwise $\hat{\mathbf{Q}}$ [Line 13]	$4d^2 r^4$
		Apply $\mathbf{Q}_0$ [Line 10]	$2dr^d s$	Apply pairwise $\hat{\mathbf{Q}}$ [Line 14]	$2d^2 r^3 s$
NE Solve [Line 11]	$2d\bar{n}r^2 + \frac{1}{3}dr^3$	Back Solve [Line 12]	$d\bar{n}r^2$	Back Solve [Line 18]	$d\bar{n}r^2$
Leading term	$4d\bar{n}rs$	Leading term	$2dr^d(s+2r) + 4d\bar{n}rs$	Leading term	$4d\bar{n}rs$

TABLE 3

Per-iteration complexity of CP-ALS algorithms for CP format input, reporting only the dominant terms for each component.

3.1. CP-ALS using Normal Equations. We use CP-ALS-NE to refer to the standard CP-ALS algorithm that uses the normal equations to solve the linear least squares problem given by (2.7) in each mode. The normal equations for this problem are given by

$$(3.1) \quad \mathbf{A}_k \mathbf{Z}_k^\top \mathbf{Z}_k = \mathbf{X}_{(k)} \mathbf{Z}_k.$$

The coefficient matrix $\mathbf{Z}_k^\top \mathbf{Z}_k$ can be computed as a Hadamard product of Gram matrices of the Khatri-Rao factors:

$$\mathbf{Z}_k^\top \mathbf{Z}_k = \mathbf{A}_d^\top \mathbf{A}_d * \dots * \mathbf{A}_{k+1}^\top \mathbf{A}_{k+1} * \mathbf{A}_{k-1}^\top \mathbf{A}_{k-1} * \dots * \mathbf{A}_1^\top \mathbf{A}_1.$$

The right hand side matrix $\mathbf{X}_{(k)} \mathbf{Z}_k$ is computed as a matricized tensor times Khatri-Rao product (MTTKRP). Pseudocode for CP-ALS-NE is given in Algorithm 3.1.

3.1.1. Dense Input. When we have a dense tensor input, the dominant cost of each subiteration comes from the MTTKRP computation of $\mathbf{M}_k = \mathbf{X}_{(k)} \mathbf{Z}_k$ in Algorithm 3.1, which has leading order arithmetic cost of $2\hat{n}^d r$. Computing $\mathbf{H}_k$ in Algorithm 3.1 requires $O(dr^2)$ operations, solving the normal equations in Algorithm 3.1 costs $O(r^3 + n_k r^2)$, and computing the Gram matrix in Algorithm 3.1 costs $O(n_k r^2)$. Thus, the sum of an entire CP-ALS-NE iteration (updating all factor matrices once) is given by

$$2d\hat{n}^d r + O(d\bar{n}r^2 + d^2 r^2 + dr^3).$$

We note that the leading order term can be reduced to $4\hat{n}^d r$ and the $O(d^2 r^2)$ term can be reduced to $O(dr^2)$ using memoization [7].

Algorithm 3.1 CP-ALS-NE

```

1: function  $\{\mathbf{A}_k\} = \text{CP-ALS-NE}(\mathcal{X}, r)$ 
2:   for  $k = 1, \dots, d$  do
3:     Initialize  $\mathbf{A}_k$ 
4:      $\mathbf{G}_k = \mathbf{A}_k^\top \mathbf{A}_k$  ▷ compute Gram
5:   end for
6:   while not converged do
7:     for  $k = 1, \dots, d$  do
8:        $\mathbf{H}_k = \mathbf{G}_d * \dots * \mathbf{G}_{k+1} * \mathbf{G}_{k-1} \dots * \mathbf{G}_1$  ▷ compute  $\mathbf{Z}_k^\top \mathbf{Z}_k$ 
9:        $\mathbf{M}_k = \mathbf{X}_{(k)} \mathbf{Z}_k$  ▷ MTTKRP
10:      Solve  $\mathbf{A}_k \mathbf{H}_k = \mathbf{M}_k$  for  $\mathbf{A}_k$  ▷ via Cholesky
11:       $\mathbf{G}_k = \mathbf{A}_k^\top \mathbf{A}_k$  ▷ update Gram
12:    end for
13:  end while
14: end function

```

195 **3.1.2. CP Format Input.** If the input tensor $\mathcal{X} = \llbracket \mathbf{B}_1, \dots, \mathbf{B}_d \rrbracket$ is already in
 196 CP format (see subsection 2.3.3), we can compute the MTTKRP more efficiently
 197 using the following identity:

$$\begin{aligned}
 \mathbf{X}_{(k)} \mathbf{Z}_k &= \mathbf{B}_k (\mathbf{B}_d \odot \dots \odot \mathbf{B}_{k+1} \odot \mathbf{B}_{k-1} \odot \dots \odot \mathbf{B}_1)^\top (\mathbf{A}_d \odot \dots \odot \mathbf{A}_{k+1} \odot \mathbf{A}_{k-1} \odot \dots \odot \mathbf{A}_1) \\
 &= \mathbf{B}_k ((\mathbf{B}_d^\top \mathbf{A}_d) * \dots * (\mathbf{B}_{k+1}^\top \mathbf{A}_{k+1}) * (\mathbf{B}_{k-1}^\top \mathbf{A}_{k-1}) * \dots * (\mathbf{B}_1^\top \mathbf{A}_1)).
 \end{aligned}$$

Algorithm 3.2 CP-Round-ALS-NE

```

1: function  $\{\mathbf{A}_k\} = \text{CP-ROUND-ALS-NE}(\{\mathbf{B}_k\}, r)$ 
2:   for  $k = 1, \dots, d$  do
3:     Initialize  $\mathbf{A}_k$ 
4:      $\mathbf{G}_k = \mathbf{A}_k^\top \mathbf{A}_k$  ▷ compute Gram
5:      $\mathbf{C}_k = \mathbf{B}_k^\top \mathbf{A}_k$  ▷ compute cross product
6:   end for
7:   while not converged do
8:     for  $k = 1, \dots, d$  do
9:        $\mathbf{H}_k = \mathbf{G}_d * \dots * \mathbf{G}_{k+1} * \mathbf{G}_{k-1} * \dots * \mathbf{G}_1$  ▷ compute  $\mathbf{Z}_k^\top \mathbf{Z}_k$ 
10:       $\mathbf{D}_k = \mathbf{B}_k (\mathbf{C}_d * \dots * \mathbf{C}_{k+1} * \mathbf{C}_{k-1} * \dots * \mathbf{C}_1)$  ▷ compute  $\mathbf{X}_{(k)} \mathbf{Z}_k$ 
11:      Solve  $\mathbf{A}_k \mathbf{H}_k = \mathbf{D}_k$  for  $\mathbf{A}_k$  ▷ via Cholesky
12:       $\mathbf{G}_k = \mathbf{A}_k^\top \mathbf{A}_k$  ▷ update Gram
13:       $\mathbf{C}_k = \mathbf{B}_k^\top \mathbf{A}_k$  ▷ update cross product
14:    end for
15:  end while
16: end function

```

200 When the input is a CP-format tensor of rank $s > r$, the cost is dominated by the
 201 computation of the right hand side of the normal equations. These costs, incurred in
 202 Algorithm 3.2, are $2n_k r s + (d-2)rs$ and $2n_k r s$ in mode k , respectively. Computing
 203 the coefficient matrix of the normal equations occurs in Algorithm 3.2 and has costs
 204 $(d-2)r^2$ and $n_k r^2$ in mode k . Solving the normal equations in Algorithm 3.2 costs
 205 $2n_k r^2 + O(r^3)$. Thus, the sum of an entire CP-Round-ALS-NE iteration (updating
 206 all factor matrices once) is given by

$$2d\bar{n}rs + 3d\bar{n}r^2 + O(d^2rs + d^2r^2 + r^3).$$

208 We note that the $O(d^2rs + d^2r^2)$ terms can be reduced to $O(drs + dr^2)$ using mem-
 209 oization. We also note that in the current implementation of this algorithm in the
 210 Tensor Toolbox [1], the $\mathbf{C}_k$ matrices are recomputed $d - 1$ times per iteration.

211 **3.2. CP-ALS using Explicit QR.** The approach proposed by Minster et al.
 212 [6], which we refer to as CP-ALS-Explicit-QR, uses the more stable QR decomposition
 213 for solving least square problem instead of the normal equations. Consider the least
 214 squares problem in mode k given by (2.7). The algorithm first performs QR on each
 215 factor matrix in $\mathbf{Z}_k$, which by Property 1 yields

$$\begin{aligned} 216 \quad \mathbf{Z}_k &= \mathbf{A}_d \odot \cdots \odot \mathbf{A}_{k+1} \odot \mathbf{A}_{k-1} \odot \cdots \odot \mathbf{A}_1 \\ 217 \quad &= (\mathbf{Q}_d \mathbf{R}_d) \odot \cdots \odot (\mathbf{Q}_{k+1} \mathbf{R}_{k+1}) \odot (\mathbf{Q}_{k-1} \mathbf{R}_{k-1}) \odot \cdots \odot (\mathbf{Q}_1 \mathbf{R}_1) \\ 218 \quad &= (\mathbf{Q}_d \otimes \cdots \otimes \mathbf{Q}_{k+1} \otimes \mathbf{Q}_{k-1} \otimes \cdots \otimes \mathbf{Q}_1)(\mathbf{R}_d \odot \cdots \odot \mathbf{R}_{k+1} \odot \mathbf{R}_{k-1} \odot \cdots \odot \mathbf{R}_1). \end{aligned}$$

Then it performs QR again on the Khatri-Rao product of triangular factors, obtaining $\mathbf{Q}_0 \mathbf{R}_0 = (\mathbf{R}_d \odot \cdots \odot \mathbf{R}_{k+1} \odot \mathbf{R}_{k-1} \odot \cdots \odot \mathbf{R}_1)$, so that $\mathbf{Z}_k = (\mathbf{Q}_d \otimes \cdots \otimes \mathbf{Q}_{k+1} \otimes \mathbf{Q}_{k-1} \otimes \cdots \otimes \mathbf{Q}_1) \mathbf{Q}_0 \mathbf{R}_0$. To solve the least square problem, we first apply the Kronecker product of orthonormal factors $\mathbf{Q}_d \otimes \cdots \otimes \mathbf{Q}_{k+1} \otimes \mathbf{Q}_{k-1} \otimes \cdots \otimes \mathbf{Q}_1$ to $\mathbf{X}_{(k)}$ from the right:

$$\mathbf{Y}_{(k)} = \mathbf{X}_{(k)}(\mathbf{Q}_d \otimes \cdots \otimes \mathbf{Q}_{k+1} \otimes \mathbf{Q}_{k-1} \otimes \cdots \otimes \mathbf{Q}_1),$$

219 which is equivalent to $\mathcal{Y} = \mathcal{X} \times_1 \mathbf{Q}_1^\top \cdots \times_{k-1} \mathbf{Q}_{k-1}^\top \times_{k+1} \mathbf{Q}_{k+1}^\top \cdots \times_d \mathbf{Q}_d^\top$, a Multi-
 220 TTM computation. Then we apply the $\mathbf{Q}_0$ to $\mathbf{Y}_{(k)}$ from the right and solve the linear
 221 system via back substitution.

Algorithm 3.3 CP-ALS-QR-Exp

```

1: function  $\{\mathbf{A}_k\} = \text{CP-ALS-QR-EXP}(\mathcal{X}, r)$ 
2:   for  $k = 1, \dots, d$  do
3:     Initialize  $\mathbf{A}_k$ 
4:      $[\mathbf{Q}_k, \mathbf{R}_k] = \text{QR}(\mathbf{A}_k)$  ▷ compute QR
5:   end for
6:   while not converged do
7:     for  $k = 1, \dots, d$  do
8:        $\mathcal{Y} = \mathcal{X} \times_1 \mathbf{Q}_1^\top \cdots \times_{k-1} \mathbf{Q}_{k-1}^\top \times_{k+1} \mathbf{Q}_{k+1}^\top \cdots \times_d \mathbf{Q}_d^\top$  ▷ Multi-TTM
9:        $[\mathbf{Q}_0, \mathbf{R}_0] = \text{QR}(\mathbf{R}_d \odot \cdots \odot \mathbf{R}_{k+1} \odot \mathbf{R}_{k-1} \odot \cdots \odot \mathbf{R}_1)$  ▷ explicit QR
10:       $\mathbf{U}_k = \mathbf{Y}_{(k)} \mathbf{Q}_0$  ▷ via matrix multiplication
11:      Solve  $\mathbf{A}_k \mathbf{R}_0^\top = \mathbf{U}_k$  for  $\mathbf{A}_k$  ▷ via triangular solve
12:       $[\mathbf{Q}_k, \mathbf{R}_k] = \text{QR}(\mathbf{A}_k)$  ▷ update QR
13:    end for
14:  end while
15: end function

```

3.2.1. Dense Input. Pseudocode for CP-ALS-QR for dense input is shown in Algorithm 3.3. For dense input, the computation of each subiteration is dominated by Algorithm 3.3, which is a multiple tensor-times-matrix (Multi-TTM) computation to form $\mathcal{Y}$ with cost $2\hat{n}^d r + O(\hat{n}^{d-1} r^2 + \cdots + \hat{n} r^d)$, where we assume the Multi-TTM is performed in order of largest mode to smallest (the largest mode is at least the geometric mean). The cost of the QR of the Khatri-Rao product of triangular factors in Algorithm 3.3 is $4r^{d+1} + O(r^3)$, where we assume $\mathbf{Q}_0$ is formed explicitly. The cost of multiplying $\mathbf{Q}_0$ by $\mathbf{Y}_{(k)}$ in Algorithm 3.3 is $2n_k r^d$. Solving the triangular

system in Algorithm 3.3 costs $n_k r^2$, and computing the QR of the factor matrix in Algorithm 3.3 is $4n_k r^2 + O(r^3)$, where again we form the orthogonal factor explicitly. Overall, the cost of updating factor matrices along all modes for CP-ALS-QR-Exp is

$$2d\hat{n}^d r + O(d\hat{n}^{d-1} r^2 + \dots + d\hat{n} r^d + d\bar{n} r^d + dr^{d+1} + d\bar{n} r^2 + dr^3).$$

222 Note that the leading order term (and several others) can be reduced by a factor of
223 $O(d)$ using memoization [4].

224 **3.2.2. CP Format Input.** In the case of a CP format tensor input, the right
225 hand side is computed more cheaply by applying Property 1 in reverse:

$$\begin{aligned} 226 \quad \mathbf{X}_{(k)}(\mathbf{Q}_d \otimes \dots \otimes \mathbf{Q}_{k+1} \otimes \mathbf{Q}_{k-1} \otimes \dots \otimes \mathbf{Q}_1) = \\ 227 \quad \mathbf{B}_k(\mathbf{Q}_d^\top \mathbf{B}_d \odot \dots \odot \mathbf{Q}_{k+1}^\top \mathbf{B}_{k+1} \odot \mathbf{Q}_{k-1}^\top \mathbf{B}_{k-1} \odot \dots \odot \mathbf{Q}_1^\top \mathbf{B}_1)^\top. \end{aligned}$$

228 The next step of applying $\mathbf{Q}_0$ to the Khatri-Rao product of cross-products $\{\mathbf{Q}_k^\top \mathbf{B}_k\}$
229 is performed using the MTTKRP computational kernel, where we treat the dense
230 matrix $\mathbf{Q}_0$ as a matricized tensor, and the final computation to compute the right
231 hand side is a small multiplication with $\mathbf{B}_k$. The algorithm is given in Algorithm 3.4.

Algorithm 3.4 CP-Round-ALS-QR-Exp

```

1: function  $\{\mathbf{A}_k\} = \text{CP-ROUND-ALS-QR-EXP}(\{\mathbf{B}_k\}, r)$ 
2:   for  $k = 1, \dots, d$  do
3:     Initialize  $\mathbf{A}_k$ 
4:      $[\mathbf{Q}_k, \mathbf{R}_k] = \text{QR}(\mathbf{A}_k)$  ▷ compute QR
5:      $\mathbf{C}_k = \mathbf{Q}_k^\top \mathbf{B}_k$  ▷ compute cross product
6:   end for
7:   while not converged do
8:     for  $k = 1, \dots, d$  do
9:        $[\mathbf{Q}_0, \mathbf{R}_0] = \text{QR}(\mathbf{R}_d \odot \dots \odot \mathbf{R}_{k+1} \odot \mathbf{R}_{k-1} \odot \dots \odot \mathbf{R}_1)$  ▷ explicit QR
10:       $\mathbf{D}_k = (\mathbf{C}_d \odot \dots \odot \mathbf{C}_{k+1} \odot \mathbf{C}_{k-1} \odot \dots \odot \mathbf{C}_1)^\top \mathbf{Q}_0$  ▷ MTTKRP
11:       $\mathbf{U}_k = \mathbf{B}_k \mathbf{D}_k$  ▷ via matrix multiplication
12:      Solve  $\mathbf{A}_k \mathbf{R}_0^\top = \mathbf{U}_k$  for  $\mathbf{A}_k$  ▷ via triangular solve
13:       $[\mathbf{Q}_k, \mathbf{R}_k] = \text{QR}(\mathbf{A}_k)$  ▷ update QR
14:       $\mathbf{C}_k = \mathbf{Q}_k^\top \mathbf{B}_k$  ▷ update cross product
15:     end for
16:   end while
17: end function

```

In the case of CP format input, the costs of Algorithm 3.4 are identical to the corresponding lines in Algorithm 3.3, with combined cost of $4r^{d+1} + 9n_k r^2 + O(r^3)$ in mode k . The cost of Algorithm 3.4, which we implement as an MTTKRP, is $2r^d s$. The cost of the matrix multiplications in Algorithm 3.4 together is $4n_k r s$. Thus, the overall cost of CP-Round-ALS-QR-Exp is

$$2dr^d s + 4dr^{d+1} + O(d\bar{n} r s + d\bar{n} r^2 + r^3).$$

232 Again, it is possible to reduce the leading order costs by a factor of $O(d)$ via memo-
233 ization.

3.3. CP-ALS using Pairwise QR. We now describe our proposed CP-ALS algorithm that uses the QR decomposition but avoids the explicit computation of the Khatri-Rao product of all $d - 1$ triangular matrices. That is, it avoids computing $\mathbf{Q}_0 \mathbf{R}_0$ as described in subsection 3.2, which has a cost that is exponential in d .

The first phase of the algorithm is the same: we perform QR decompositions on individual factor matrices and exploit Property 1:

$$\mathbf{Z}_k = (\mathbf{Q}_d \otimes \cdots \otimes \mathbf{Q}_{k+1} \otimes \mathbf{Q}_{k-1} \otimes \cdots \otimes \mathbf{Q}_1)(\mathbf{R}_d \odot \cdots \odot \mathbf{R}_{k+1} \odot \mathbf{R}_{k-1} \odot \cdots \odot \mathbf{R}_1).$$

To compute the QR decomposition of the Khatri-Rao product of triangular factors, we perform a sequence of pairwise reductions. While the order is arbitrary, for notational convenience we start from the right: we compute a QR decomposition of an explicit Khatri-Rao product of the last two triangular factors, $\mathbf{R}_2 \odot \mathbf{R}_1 = \hat{\mathbf{Q}}_1 \hat{\mathbf{R}}_2$, yielding

$$\begin{aligned} \mathbf{R}_{d-1} \odot \cdots \odot \mathbf{R}_{k+1} \odot \mathbf{R}_{k-1} \odot \cdots \odot \hat{\mathbf{Q}}_1 \hat{\mathbf{R}}_2 = \\ (\mathbf{I}_{r^{d-2}} \otimes \hat{\mathbf{Q}}_1)(\mathbf{R}_{d-1} \odot \cdots \odot \mathbf{R}_{k+1} \odot \mathbf{R}_{k-1} \odot \cdots \odot \hat{\mathbf{R}}_2). \end{aligned}$$

We then repeat the pairwise QR decomposition on two rightmost remaining factors $\mathbf{R}_3 \odot \hat{\mathbf{R}}_2$, and so on (skipping the k th factor), each time reducing the number of triangular factors by one. After $d - 2$ pairwise QR decompositions, we obtain the QR decomposition of $\mathbf{Z}_k$ (with implicit orthogonal factor):

$$\mathbf{Z}_k = (\mathbf{Q}_d \otimes \cdots \otimes \mathbf{Q}_{k+1} \otimes \mathbf{Q}_{k-1} \otimes \cdots \otimes \mathbf{Q}_1)(\mathbf{I}_{r^{d-2}} \otimes \hat{\mathbf{Q}}_1)(\mathbf{I}_{r^{d-3}} \otimes \hat{\mathbf{Q}}_2) \cdots (\mathbf{I}_r \otimes \hat{\mathbf{Q}}_{d-2}) \hat{\mathbf{Q}}_{d-1} \mathbf{R}.$$

As in the previous approach, applying the orthogonal factor to the unfolding of the tensor $\mathbf{X}_{(k)}$ starts with a Multi-TTM operation:

$$\mathcal{Y} = \mathcal{X} \times_1 \mathbf{Q}_1^\top \cdots \times_{k-1} \mathbf{Q}_{k-1}^\top \times_{k+1} \mathbf{Q}_{k+1}^\top \cdots \times_d \mathbf{Q}_d^\top.$$

Applying the orthogonal factors computed during the pairwise phase differs depending on the structure of the input.

3.3.1. Dense Input. In the case the input $\mathcal{X}$ is dense, $\mathcal{Y}$ is a dense tensor with dimensions $r \times \cdots \times r \times n_k \times r \times \cdots \times r$. Each $\hat{\mathbf{Q}}_\ell$ (for $1 \leq \ell < d$ and $\ell \neq k$) has dimension $r^2 \times r$, and postmultiplying $\mathbf{Y}_{(k)}$ by the Kronecker product of an orthogonal factor and the identity corresponds to a contraction with two of the modes of the tensor $\mathbf{Y}$. To simplify the presentation of the approach in Algorithm 3.5, we first permute the tensor so that the k th mode is last, which allows the contraction to be performed on the first two modes each time.

The cost of the Multi-TTM in Algorithm 3.5 is the same as in Algorithm 3.3: $2\hat{n}^d r + O(\hat{n}^{d-1} r^2 + \cdots + \hat{n} r^d)$. The permutation in Algorithm 3.5 has no arithmetic cost, though in practice it may take significant time due to the high relative cost of memory movement. The pairwise QR decompositions (Algorithm 3.5) together cost $4(d-2)r^4 + O(dr^3)$ for mode k , and applying the pairwise orthogonal factors via TTM (Algorithm 3.5) costs a total of $2n_k r^d + 2n_k r^{d-1} + \cdots + 2n_k r^3$. The triangular solve in Algorithm 3.5 costs $n_k r^2$. The cost of the QR decomposition in Algorithm 3.5 is $4n_k r^2 + O(r^3)$ in mode k . Thus, the total cost of a full iteration (updating each factor matrix once) of CP-ALS-QR-Imp is

$$2d\hat{n}^d r + 2d\bar{n} r^d + 4d^2 r^4 + O(d\hat{n}^{d-1} r^2 + d\bar{n} r^{d-1}).$$

The leading order cost can be reduced by a factor of $O(d)$ via memoization [4].

Algorithm 3.5 CP-ALS-QR-Imp

```

1: function  $\{\mathbf{A}_k\} = \text{CP-ALS-QR-IMP}(\mathcal{X}, r)$ 
2:   for  $k = 1, \dots, d$  do
3:     Initialize  $\mathbf{A}_k$ 
4:      $[\mathbf{Q}_k, \mathbf{R}_k] = \text{QR}(\mathbf{A}_k)$  ▷ compute QR
5:   end for
6:   while not converge do
7:     for  $k = 1, \dots, d$  do
8:        $\mathcal{Y} = \mathcal{X} \times_1 \mathbf{Q}_1^\top \cdots \times_{k-1} \mathbf{Q}_{k-1}^\top \times_{k+1} \mathbf{Q}_{k+1}^\top \cdots \times_d \mathbf{Q}_d^\top$  ▷ Multi-TTM
9:        $\mathcal{Y} = \text{PERMUTE}(\mathcal{Y}, [1, \dots, k-1, k+1, \dots, d, k])$  ▷  $\mathcal{Y}$  is  $r \times \dots \times r \times n_k$ 
10:       $\mathbf{R} = \mathbf{R}_1$ 
11:      for  $\ell = 2, \dots, d-1$  do
12:         $[\hat{\mathbf{Q}}, \mathbf{R}] = \text{QR}(\mathbf{R}_\ell \odot \mathbf{R})$  ▷ pairwise QR
13:         $\mathcal{Y} = \text{RESHAPE}(\mathcal{Y}, [r^2, r^{d-\ell-1}, n_k])$  ▷ linearize 1st two modes
14:         $\mathcal{Y} = \mathcal{Y} \times_1 \hat{\mathbf{Q}}^\top$  ▷ TTM
15:      end for
16:      Solve  $\mathbf{A}_k \mathbf{R}^\top = \mathbf{Y}_{(1)}^\top$  for  $\mathbf{A}_k$  ▷ via triangular solve
17:       $[\mathbf{Q}_k, \mathbf{R}_k] = \text{QR}(\mathbf{A}_k)$  ▷ update QR
18:    end for
19:  end while
20: end function

```

3.3.2. CP Format Input. With CP format input, we can use the Khatri-Rao structure $\mathbf{X}_{(k)} = \mathbf{B}_k(\mathbf{B}_d \odot \cdots \odot \mathbf{B}_{k+1} \odot \mathbf{B}_{k-1} \odot \cdots \odot \mathbf{B}_1)^\top$ when applying the orthogonal factor into $\mathbf{X}_k$. As described in subsection 3.2.2, this gives us

$$\begin{aligned} \mathbf{Y}_k &= \mathbf{B}_k(\mathbf{Q}_d^\top \mathbf{B}_d \odot \cdots \odot \mathbf{Q}_{k+1}^\top \mathbf{B}_{k+1} \odot \mathbf{Q}_{k-1}^\top \mathbf{B}_{k-1} \odot \cdots \odot \mathbf{Q}_1^\top \mathbf{B}_1)^\top \\ &= \mathbf{B}_k(\mathbf{C}_d \odot \cdots \odot \mathbf{C}_{k+1} \odot \mathbf{C}_{k-1} \odot \cdots \odot \mathbf{C}_1)^\top. \end{aligned}$$

For the pairwise updates, we continue to exploit the Khatri-Rao structure. To apply the first pairwise orthogonal factor, we use the identity

$$\begin{aligned} (\mathbf{C}_d \odot \cdots \odot \mathbf{C}_{k+1} \odot \mathbf{C}_{k-1} \odot \cdots \odot \mathbf{C}_3 \odot \mathbf{C}_2 \odot \mathbf{C}_1)^\top (\mathbf{I}_{r^{d-2}} \otimes \hat{\mathbf{Q}}_1) &= \\ (\mathbf{C}_d \odot \cdots \odot \mathbf{C}_{k+1} \odot \mathbf{C}_{k-1} \odot \cdots \odot \mathbf{C}_3 \odot \hat{\mathbf{Q}}_1^\top (\mathbf{C}_2 \odot \mathbf{C}_1))^\top &= \\ \mathbf{C}_d \odot \cdots \odot \mathbf{C}_{k+1} \odot \mathbf{C}_{k-1} \odot \cdots \odot \mathbf{C}_3 \odot \hat{\mathbf{C}}_2. \end{aligned}$$

That is, we form the explicit Khatri-Rao product of the last two factors (forming a dense matrix with dimensions $r^2 \times s$) and apply the pairwise orthogonal factor using matrix multiplication, which reduces the number of $r \times s$ Khatri-Rao factors by one. In the next step, we apply the next $r^2 \times r$ pairwise orthogonal factor to the last two remaining factors, and so on. After $d-2$ steps, we obtain a final $r \times s$ factor that is multiplied into $\mathbf{B}_k$ to form the right hand side of the triangular linear system. The pseudocode for this approach, where we apply the pairwise orthogonal factors as soon as we compute them, is given in Algorithm 3.6.

For CP format input of rank s , we first consider the costs of the factor matrix QR decompositions and applying the individual orthogonal factors. In mode k , the cost of the QR in Algorithm 3.6 is $4n_k r^2$, and the cost of applying the explicit orthogonal factor to the corresponding input factor matrix in Algorithm 3.6 is $2n_k r s$. Performing

Algorithm 3.6 CP-Round-ALS-QR-Imp

```

1: function  $\{\mathbf{A}_k\} = \text{CP-ROUND-ALS-QR-IMP}(\{\mathbf{B}_k\}, r)$ 
2:   for  $k = 1, \dots, d$  do
3:     Initialize  $\mathbf{A}_k$ 
4:      $[\mathbf{Q}_k, \mathbf{R}_k] = \text{QR}(\mathbf{A}_k)$  ▷ compute QR
5:      $\mathbf{C}_k = \mathbf{Q}_k^\top \mathbf{B}_k$  ▷ compute cross product
6:   end for
7:   while not converge do
8:     for  $k = 1, \dots, d$  do
9:       if  $k \neq 1$  then  $a = 1$  else  $a = 2$  ▷ set starting index
10:       $\mathbf{R} = \mathbf{R}_a, \mathbf{D}_k = \mathbf{C}_a$  ▷ initialize pairwise results
11:      for  $\ell = 1, \dots, d$  do
12:        if  $\ell \neq k$  and  $\ell \neq a$  then
13:           $[\hat{\mathbf{Q}}, \mathbf{R}] = \text{QR}(\mathbf{R}_\ell \odot \mathbf{R})$  ▷ pairwise QR
14:           $\mathbf{D}_k = (\mathbf{C}_\ell \odot \mathbf{D}_k)^\top \hat{\mathbf{Q}}$  ▷ pairwise apply
15:        end if
16:      end for
17:       $\mathbf{U}_k = \mathbf{B}_k \mathbf{D}_k$  ▷ via matrix multiplication
18:      Solve  $\mathbf{A}_k \mathbf{R}_0^\top = \mathbf{U}_k$  for  $\mathbf{A}_k$  ▷ via triangular solve
19:       $[\mathbf{Q}_k, \mathbf{R}_k] = \text{QR}(\mathbf{A}_k)$  ▷ update QR
20:       $\mathbf{C}_k = \mathbf{Q}_k^\top \mathbf{B}_k$  ▷ update cross product
21:    end for
22:  end while
23: end function

```

the pairwise QR decompositions and applying the explicit orthogonal factors occurs in Algorithm 3.6. For each subiteration, the cost of the two operations, which are both performed $d - 2$ times, is $4dr^4 + 2dr^3s + O(r^4)$. The final multiplication and triangular solve in Algorithm 3.6 together cost $2n_krs + n_kr^2$. Performing a full iteration and updating each factor matrix once has total cost

$$4d\bar{n}rs + 5d\bar{n}r^2 + 2d^2r^3s + 4d^2r^4 + O(r^4).$$

272 The terms involving factors of d^2 can likely be reduced by a factor of d via memoiza-
 273 tion, but we do not consider that optimization here.

274 4. Experimental Results.

275 **4.1. Sine of Sums Tensor.** To test the accuracy and efficiency of the QR-based
 276 method, we use a discretization of sine of sums: $\sin(x_1 + \dots + x_d)$. Repeated using
 277 the textbook sine of sums and cosine of sums identities for two variables, we obtain a
 278 2^{d-1} -term separated representation of the function. For example,

$$\begin{aligned}
 \sin(x_1 + x_2 + x_3) &= \sin(x_1) \cos(x_2) \cos(x_3) + \cos(x_1) \cos(x_2) \sin(x_3) \\
 &\quad + \cos(x_1) \sin(x_2) \cos(x_3) - \sin(x_1) \sin(x_2) \sin(x_3).
 \end{aligned}$$

280 As described in previous work [2, 6], there exists a d -term separated representation
 281 of the form

$$(4.1) \quad \sin\left(\sum_{j=1}^d x_j\right) = \sum_{j=1}^d \sin(x_j) \prod_{k=1, k \neq j}^d \frac{\sin(x_k - \alpha_k - \alpha_j)}{\sin(\alpha_k - \alpha_j)}$$

for all choices of α_j such that $\sin(\alpha_k - \alpha_j) \neq 0$ for all $j \neq k$. Our goal is to compute this much more efficient d -term representation using the 2^{d-1} -term representation numerically using discretized representations of the functions. This corresponds to running CP-ALS with CP format input.

4.2. Algorithm Comparison for Single Least Squares Problem. We first compare the three methods for solving a single least squares problem where the coefficient matrix is a Khatri-Rao product and the right hand side has matching Khatri-Rao structure, as given in (2.8):

$$\begin{aligned} \min_{\mathbf{A}_k} \|\mathbf{B}_k \mathbf{Y}_k^\top - \mathbf{A}_k \mathbf{Z}_k^\top\|_F, \text{ where} \\ \mathbf{Y}_k = \mathbf{B}_N \odot \cdots \odot \mathbf{B}_{k+1} \odot \mathbf{B}_{k-1} \odot \cdots \odot \mathbf{B}_1 \text{ and} \\ \mathbf{Z}_k = \mathbf{A}_N \odot \cdots \odot \mathbf{A}_{k+1} \odot \mathbf{A}_{k-1} \odot \cdots \odot \mathbf{A}_1. \end{aligned}$$

Here we set the right hand side factors to be discretizations of the 2^{d-1} -term representation, using an n -point discretization of $[0, 2\pi)$. This implies that each $\mathbf{B}_j$ has dimension $n \times 2^{d-1}$. We set the coefficient matrix $\mathbf{Z}_k$ to be a Khatri-Rao product of matrices corresponding to n -point discretizations of the d -term representation so that each $\mathbf{A}_j$ has dimension $n \times d$. We choose α_j 's to be equally spaced within $[0, 2\pi)$. Leaving one factor matrix $\mathbf{A}_k$ out of the representation, our goal is to recompute it by solving the least squares problem.

Table 4 reports the accuracy of the three methods as we vary the condition number as measured by the relative forward error $\|\mathbf{A}_k - \hat{\mathbf{A}}_k\|/\|\mathbf{A}_k\|$, where $\mathbf{A}_k$ is the true solution given by the discretization of (4.1) and $\hat{\mathbf{A}}_k$ is the computed solution. We adjust the condition number of the coefficient matrix by varying d and the distance between consecutive parameters α_j and α_{j+1} , and we fix the discretization parameter $n = 50$. We observe that as the distance between consecutive parameters decreases, the condition number of the coefficient matrix increases, and this effect is more pronounced for $d = 5$ than $d = 3$. As the condition number increases, we see that the forward error of all methods increases, as expected. However, this loss of accuracy is much worse for the normal equations method, and when the condition number is as large as the reciprocal of the square root of machine precision, it completely breaks down. For the QR-based methods, the loss of accuracy is much more graceful, and both methods maintain forward error at or below the square root of machine precision even when the condition number is on the order of $1e13$. This experiment confirms the difference in theoretical guarantees presented in (2.5) and (2.6). We note that the implicit Pairwise QR method yields slightly more accurate results than the explicit QR method, which we conjecture is due to the computations involving explicit vectors of dimension n^{d-1} , which the implicit method avoids.

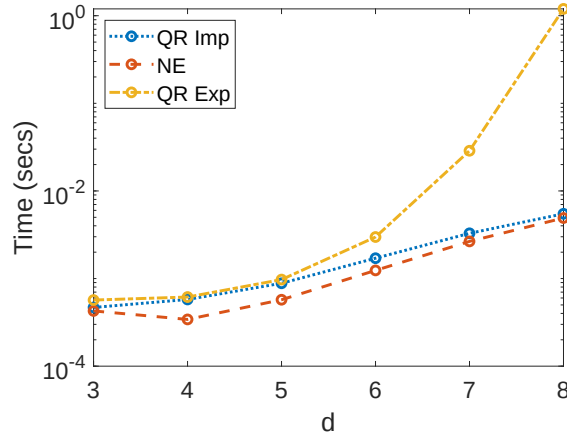
Figure 1 reports the total time for solving the single least squares problem varying the number of modes d and fixing the discretization parameter $n = 2000$. Here we observe the exponential dependence on d of the explicit QR method's complexity: as d increases, the time required to solve the least square problem increases drastically. The normal equations method and the implicit QR method, with complexities that are linear in d , have running times that increase much more slowly with d and are comparable to each other.

In Figure 2, we see a breakdown of the running times for the normal equations and implicit QR methods as reported in Figure 1. Our first observation is that the time spent in applying the factor-wise orthonormal matrices to the right hand side and computing the right hand side of the normal equations, labeled $Q^T B / A^T B$,

d	$ \alpha_j - \alpha_{j+1} $	$\kappa(\mathbf{Z}_k)$	NE Err	QR Exp Err	QR Imp Err
3	1e00	1.51e00	3.64e-16	1.44e-15	5.07e-16
3	1e-1	5.02e02	6.12e-14	3.70e-15	5.74e-16
5	1e-1	7.38e03	9.80e-11	9.70e-15	1.85e-15
3	1e-2	4.77e05	1.32e-08	5.40e-13	8.99e-15
5	1e-2	3.71e08	3.51e-01	4.87e-12	1.27e-14
3	1e-4	4.77e11	8.89e-01	8.43e-09	1.31e-12
5	1e-3	3.64e13	1.00e00	9.74e-07	1.94e-09

TABLE 4

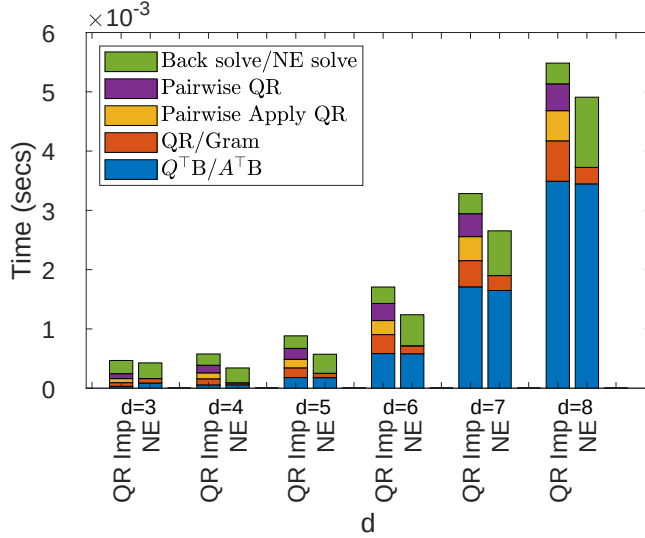
Comparison of relative forward error when varying d and $|\alpha_j - \alpha_{j+1}|$ for single least squares problem given by sine of sums

FIG. 1. Comparison of run time when varying d for NE, QR Imp, and QR Exp ($n = 2000$)

dominates the running time and is nearly identical between algorithms. This confirms the argument made in subsection 2.2 for unstructured least squares problems with many right hand sides. The second observation is that the time spent in factor-wise QR's is greater than that of computing Gram matrices, as they require twice the flops and are slightly less efficient. The pairwise QR and the application of the pairwise orthonormal factors is a unique cost to the implicit QR method, and that time is the main reason that the implicit QR method is slightly slower than the normal equations method. We also note that the linear system solve is noticeably slower for the normal equations method; we conjecture this is because Matlab is resorting to an SVD to solve a linear system it has detected is ill conditioned.

4.3. CP-ALS Algorithm Comparisons. In the next set of experiments, we incorporate the implicit Pairwise QR method into CP-ALS and further compare the runtime and accuracy with CP-Round-ALS-NE and CP-Round-ALS-QR-Exp. We use a discretized version of the 2^{d-1} -term representation of the sine of sums tensor as input, and we initialize CP-ALS randomly.

Figure 3 reports the per-iteration run time of each of the three methods, varying $d \in \{3, 5, 7, 9\}$ and $n \in \{10000, 15000, 30000\}$. For 3-way tensors, the running times are comparable, though the normal equations is typically the fastest. For larger d , we see that the explicit QR method's running time increases drastically due to the

FIG. 2. Breakdown of run time when varying d for NE and QR Imp ($n = 2000$)

exponential dependence on d , and we did not attempt to complete the experiment for $d = 9$. While the normal equations and implicit QR methods have comparable time for $d = 5$, we see that the implicit QR method outperforms NE for $d = 7$ and $d = 9$. This improvement is because we use the Tensor Toolbox's implementation of CP-ALS, which performs redundant computation in the MTTKRPCs. We avoid this in our implementation of the QR-based methods, using memoization. We note that it is possible to apply memoization to the normal equations based method, and we would expect to see comparable performance to the implicit QR method with that optimization.

To compare the accuracy of the NE-based and implicit Pairwise QR-based CP-ALS algorithms, we perform experiments with the sine of sums input with varying numbers of modes $d \in \{5, 7, 10\}$ and fixed $n = 10$. Figure 4 shows the convergence behavior as measured by the relative backward/residual error over 40 iterations for two different random initializations. We observe that overall, the QR-based method is able to achieve lower error in each of the 6 experiments. In the case of $d = 5$ and $d = 7$, we observed little variability over initialization: the QR-based method converged to errors that were orders of magnitude smaller than the NE-based method, and that difference grows with d . In the case of $d = 10$, we observed differing behavior for different initializations, confirming previously reported observations [6]: the NE-based method failed about 80% of the time (as shown in the 2nd trial) and achieved moderately large error about 20% of the time (as shown in the 1st trial). The QR-based method, on the other hand, consistently converged to solutions with small residual error, illustrating its better numerical stability properties.

4.4. Accuracy and Running Time Tradeoff using Single and Double Precision. In these experiments, we aim to evaluate the trade-off between accuracy and running time for NE-based and implicit Pairwise QR-based CP-ALS algorithms, using both single and double precision floating-point representations. We perform experiments on synthetic tensors where the factor matrices (and their Khatri-Rao products) are ill-conditioned. The synthetic tensor is generated using the method

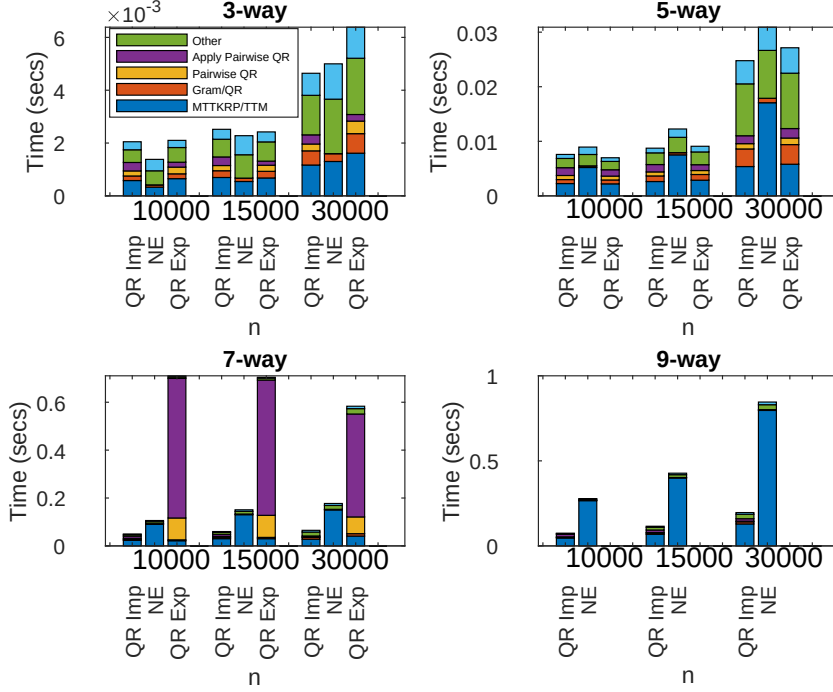


FIG. 3. Comparison of per-iteration run time when varying d and n for NE, QR Imp and QR Exp

from [6], with Gaussian noise added to factor matrices of specified condition numbers. Each factor matrix $\mathbf{A}_j$ is created as $\mathbf{A}_j = \mathbf{U}_j \mathbf{\Sigma} \mathbf{V}^T$, where $\mathbf{U}_j \in \mathbb{R}^{n_j \times R}$ is a random matrix with orthonormal columns, $\mathbf{\Sigma} \in \mathbb{R}^{R \times R}$ is a diagonal matrix with decreasing singular values, ensuring the condition number of $\mathbf{A}_j$ is a predetermined value κ , and $\mathbf{V} \in \mathbb{R}^{R \times R}$ is an orthogonal matrix shared by all factor matrices $\mathbf{A}_j$ for $j = 1, \dots, d$. This construction allows us to control the condition number κ of each factor matrix and of the Khatri–Rao product of $d - 1$ factor matrices.

In our experiments, we first generate a three-way CP tensor $\mathcal{Y}$ of size $10000 \times 10000 \times 10000$ with CP-rank 50, using the procedure described above, and set the condition number $\kappa = 10^{10}$. To this tensor, we add a noise tensor $\mathcal{N}$ whose factor matrices have 450 columns and are initialized with i.i.d. samples from the standard normal distribution with noise level $\eta = 10^{-5}$. The formal rank of the resulting CP-tensor ($\mathcal{X} = \mathcal{Y} + \eta \mathcal{N}$) becomes 500, the sum of the ranks of $\mathcal{Y}$ and $\mathcal{N}$ due to the addition of noise, but it is accurately approximated by a rank-50 CP tensor. We apply the NE-based and implicit Pairwise QR-based CP-ALS algorithms using both single and double-precision floating-point representations to compress it to rank 50. We set the maximum number of iterations to 20 and use a convergence tolerance of 10^{-15} for the relative error. We use a random guess for initialization, and all algorithms start with the same initial factor matrices. Figure 5 summarizes our experimental results.

From Figure 5, we observe that running time of NE-based and implicit QR-based CP-ALS algorithms in single precision are comparable and shortest. Note that the

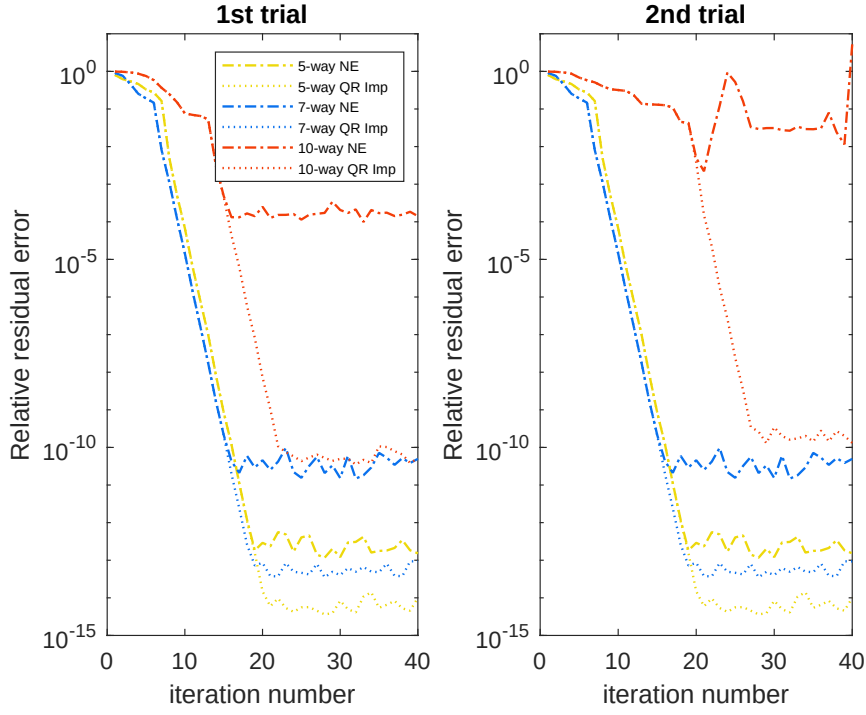


FIG. 4. Comparison of convergence behavior of NE and QR Imp for varying d and random initializations ($n = 10$)

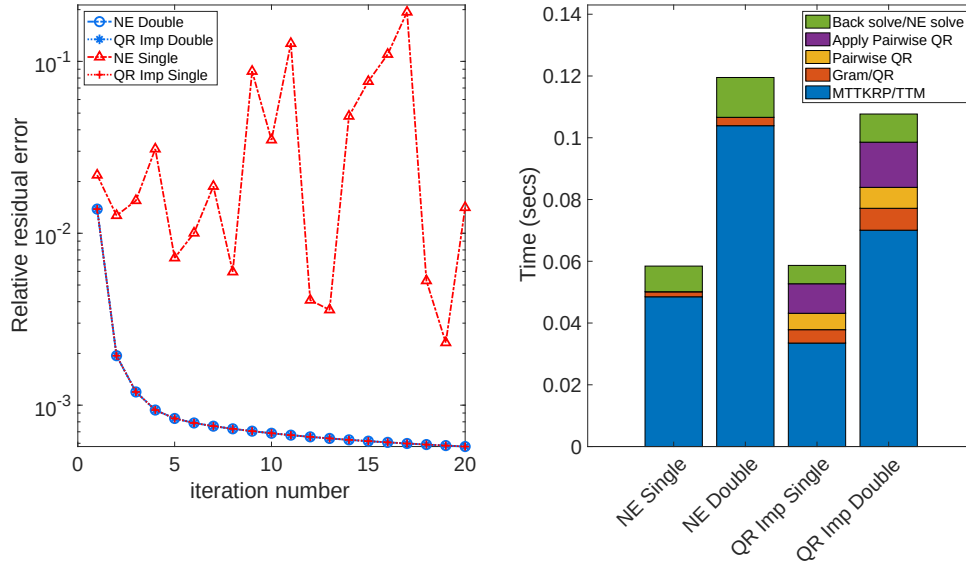


FIG. 5. Comparison of accuracy and running time of NE and QR Imp based CP-ALS algorithms in single and double precision.

leading-order costs of each algorithm (as given in Table 3), correspond to the MT-TKRP/TTM computations (given in blue). While the complexity analysis suggests they should be comparable between NE-based and QR-based algorithm, the NE-based implementation we benchmark is that of the Tensor Toolbox, which does not use memoization of the cross products as shown in line 13 of Algorithm 3.2. However, the NE-based CP-ALS algorithm in single precision exhibits numerical instability, leading to high relative residual errors and erratic convergence behavior. In contrast, the implicit QR-based CP-ALS algorithm in single precision remains numerically stable, resulting in consistent accuracy and stable behavior, similar to its double-precision counterparts. The single-precision QR-based algorithm is approximately $2\times$ faster than either method in double precision. These results show that the QR-based method in single precision offers a good trade-off between stability and efficiency and makes them a more reliable and faster alternative to the NE-based method in double precision for many practical applications.

5. Conclusion. In this work, we implement a new version of CP-ALS algorithm with QR decomposition that in the case of CP-format input reduces the exponential time complexity of a previously proposed solution [6] to polynomial time without sacrificing numerical stability. The time complexity roughly matches that of the standard normal equations based CP-ALS algorithm as implemented in the Tensor Toolbox [1]. As evidenced by our theoretical and experimental analysis, the algorithm simultaneously achieves numerical stability and computational efficiency.

Our experiments with a single least squares problem show that if the problem is very ill conditioned (condition number of $4e13$), our algorithm achieves an error that is 9 orders of magnitude smaller than the normal equations approach. Likewise, for an 8-way tensor, we observe a speedup of about $100\times$ over the previously proposed QR-based method. In experiments using CP-ALS to perform CP rounding, we observe a $14\times$ speedup over the previous QR-based method on a 7-way tensor and $4\times$ speedup over Tensor Toolbox’s implementation of the NE-based method. Further, we also see convergence to more accurate solutions when using our method for 5-way, 7-way, and 10-way tensors correspond to discovery of a sine of sums identity. We also show that the use of a more stable algorithm enables users to reduce the working precision while maintaining the stability of convergence and accuracy of the final result. Switching from double to single precision provides another $2\times$ speedup.

Our future work includes implementing this QR-based method within Tensor Toolbox, providing a more accurate CP-ALS approach for CP-format input that does not sacrifice computational efficiency. We also note that there are more opportunities to exploit memoization in our pairwise QR algorithm. Finally, we believe that the pairwise QR approach can improve the cost and accuracy for computing leverage scores for randomized methods as described in [3], which we would like to explore further.

REFERENCES

- [1] B. W. BADER, T. G. KOLDA, ET AL., *Tensor toolbox for MATLAB*, 2023, www.tensortoolbox.org. Version 3.6.
- [2] G. BEYLKIN AND M. J. MOHLENKAMP, *Numerical operator calculus in higher dimensions*, Proceedings of the National Academy of Sciences, 99 (2002), pp. 10246–10251, <https://doi.org/10.1073/pnas.112329799>.
- [3] V. BHARADWAJ, O. A. MALIK, R. MURRAY, L. GRIGORI, A. BULUC, AND J. DEMMEL, *Fast exact leverage score sampling from khatri-rao products with applications to tensor decomposition*, Advances in Neural Information Processing Systems, 36 (2024), <https://proceedings.neurips>.

- cc/paper_files/paper/2023/file/959f70ee50044bed305e48e3484005a7-Paper-Conference.pdf.
- [4] O. KAYA AND Y. ROBERT, *Computing dense tensor decompositions with optimal dimension trees*, *Algorithmica*, 81 (2019), pp. 2092–2121, <https://doi.org/10.1007/s00453-018-0525-3>.
- [5] T. G. KOLDA AND B. W. BADER, *Tensor decompositions and applications*, *SIAM Review*, 51 (2009), pp. 455–500, <https://doi.org/10.1137/07070111X>.
- [6] R. MINSTER, I. VIVIANO, X. LIU, AND G. BALLARD, *CP decomposition for tensors via alternating least squares with qr decomposition*, *Numerical Linear Algebra with Applications*, 30 (2023), p. e2511, <https://doi.org/10.1002/nla.2511>.
- [7] A. H. PHAN, P. TICHAVSKÝ, AND A. CICHOCKI, *Fast alternating ls algorithms for high order candecomp/parafac tensor factorizations*, *IEEE Transactions on Signal Processing*, 61 (2013), pp. 4834–4846, <https://doi.org/10.1109/TSP.2013.2269903>.
- [8] L. N. TREFETHEN AND D. BAU III, *Numerical Linear Algebra*, vol. 50, SIAM, 1997.